

Worksheet — 1.4 Data Types, Data Structures & Boolean Algebra — ANSWER SHEET

Separate answer sheet / mark scheme — keep for marking.

Answer key

Activity 1 — Conversions

#	Denary	Binary (8-bit)	Hex
1	217	11011001	D9
2	173	10101101	AD
3	90	01011010	5A
4	202	11001010	CA
5	47	00101111	2F
6	190	10111110	BE
7	45	00101101	2D
8	179	10110011	B3

Worked examples: - **217**: $128+64=192$, $+16=208$, $+8=216$, $+1=217 \rightarrow 11011001$. Nibbles $1101\ 1001 = D9$. - **10101101**: $128+32+8+4+1 = 173$. Nibbles $1010\ 1101 = AD$. - **5A**: $5=0101$, $A=1010 \rightarrow 01011010 = 64+16+8+2 = 90$. - **202**: $128+64=192$, $+8=200$, $+2=202 \rightarrow 11001010 = 1100\ 1010 = CA$. - **00101111**: $32+8+4+2+1 = 47$. $0010\ 1111 = 2F$. - **190**: $128+32=160$, $+16=176$, $+8=184$, $+4=188$, $+2=190 \rightarrow 10111110 = BE$. - **2D**: $2=0010$, $D=1101 \rightarrow 00101101 = 32+8+4+1 = 45$. - **10110011**: $128+32+16+2+1 = 179$. $1011\ 0011 = B3$.

Activity 2 — Two's complement

(a)

Denary	8-bit two's complement	Working
-20	11101100	$+20 = 00010100$; invert 11101011 ; $+1 = 11101100$
-100	10011100	$+100 = 01100100$; invert 10011011 ; $+1 = 10011100$
-1	11111111	$+1 = 00000001$; invert 11111110 ; $+1 = 11111111$

Check -20: $-128+64+32+8+4 = -20 \checkmark$. -100: $-128+16+8+4 = -100 \checkmark$.

(b)(i) $35 - 18 = 17$

```

A   = 00100011   (35)
-B  = 11101110   (-18: +18=00010010 → invert 11101101 → +1 = 11101110)
-----
      1 00010001   carry out of MSB = 1 → DISCARD
Sum  = 00010001

```

Denary result = **+17** (MSB 0 = positive; $16+1 = 17$) ✓

(b)(ii) 20 - 50 = -30

```

A   = 00010100   (20)
-B  = 11001110   (-50: +50=00110010 → invert 11001101 → +1 = 11001110)
-----
Sum  = 11100010   (no carry out of MSB)

```

Denary result = **-30**. The MSB is **1**, so the result is negative. Check: $-128+64+32+2 = -30$ ✓.

Activity 3 — Floating-point normalisation

Given mantissa **0.0001101** (point after first bit). - **(a)** Move the point **3 places to the right** so the mantissa becomes **0.1101...** (first bit after the point is 1). - **(b)** Moving the point **right** ⇒ **exponent decreases**. - **(c)** Normalised mantissa = **0.1101000** (**0.1101** with trailing zero padding). - **(d)** Original exponent was effectively 0; moving right 3 places means we must **subtract 3** → exponent = **-3**. (Value = **0.0001101** × $2^0 = 0.1101 \times 2^{-3}$, unchanged.) - **(e)** Normalisation removes wasted leading zeros so the fixed-width mantissa holds the **maximum number of significant figures** → **greatest precision** (and gives a unique representation).

Activity 4 — Binary shifts

(a) **00010110** logical left 2 = **01011000**. Effect: shift **left**, multiplies by $2^2 = \times 4$. (Check: $22 \times 4 = 88 = 01011000$ ✓.)

(b) **10110000** logical right 3 = **00010110**. Effect: shift **right**, divides by $2^3 = \div 8$ (integer division; bits shifted off are lost, 0s filled from the left). (Check: $176 \div 8 = 22 = 00010110$ ✓.)

Activity 5 — Data structures

(a) 1 → **B**, 2 → **G**, 3 → **J**, 4 → **C**, 5 → **F**, 6 → **H**, 7 → **A**, 8 → **D**, 9 → **I**, 10 → **E**.

(b) Trace table (stack = LIFO, removes the **top/most recent**; queue = FIFO, removes the **front/oldest**):

Step	Operation	Stack (top → bottom)	Stack removed	Queue (front → rear)	Queue removed
1	add 7	7	—	7	—
2	add 3	3, 7	—	7, 3	—
3	add 9	9, 3, 7	—	7, 3, 9	—
4	remove	3, 7	9	3, 9	7
5	add 5	5, 3, 7	—	3, 9, 5	—
6	remove	3, 7	5	9, 5	3

Activity 6 — Boolean

(a) XOR

A	B	Q
0	0	0
0	1	1
1	0	1
1	1	0

NAND

A	B	Q
0	0	1
0	1	1
1	0	1
1	1	0

(b)(i) $\neg(A \cdot B) + B = (\neg A + \neg B) + B$ (De Morgan's on the bracket) = $\neg A + (\neg B + B)$ (associative/commutative) = $\neg A + 1$ (complement: $\neg B + B = 1$) = **1** (null law). The expression is always true.

(b)(ii) $\neg(\neg A + \neg B) = \neg(\neg A) \cdot \neg(\neg B)$ (De Morgan's: break the bar, swap OR $\rightarrow$ AND) = $A \cdot B$ (double negation on each term) = **$A \cdot B$** .

(c) Half adder

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Sum = $A \oplus B$ **Carry** = $A \cdot B$.

Challenge box

(a) 10000000 = the -128 column only set, all others 0 $\rightarrow$ **-128**. (It is not zero; zero is 00000000 .)

(b) In two's complement the MSB is a single negative weight, so each bit pattern maps to exactly one value; 00000000 is the only pattern that sums to 0. Sign-and-magnitude has both 00000000 (+0) and 10000000 (-0); two's complement avoids this, giving **one** zero (and one extra negative value, -128).

(c) Green $90_{16} = (9 \times 16) + 0 = \mathbf{144}$. Blue $FF_{16} = (15 \times 16) + 15 = \mathbf{255}$. Hex is used because each pair of hex digits = exactly one byte (8 bits / one colour channel), making it a **compact, human-readable, lossless** shorthand for the underlying binary, far less error-prone than writing 24 raw bits.